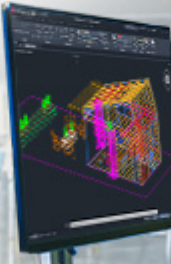


신뢰할 수 있는 솔루션

AutoCAD는 모든 프로젝트에서 정확성과 효율성을 보장합니다.

무료 체험판





검색어를 입력하세요.

📖 점프 투 자바 (/book/31)
00장 들어가기 전에
00-01 머리말
00-02 주요변경이력
00-03 저자소개
00-04 책 구입 안내
01장 자바란 무엇인가?
01-01 자바란?
01-02 자바로 무엇을 할 수 있을까?
01-03 자바 둘러보기
01-04 자바의 8가지 특징
02장 자바 시작하기
02-01 자바 코드의 구조 살펴보기
02-02 변수와 자료형

02-03 이름 짓는 규칙
02-04 주석이란?
03장 자바의 기초 - 자료형
03-01 숫자
03-02 불
03-03 문자
03-04 문자열
03-05 StringBuffer
03-06 배열
03-07 리스트
03-08 맵
03-09 집합
03-10 상수 집합
03-11 형 변환과 final
03장 되새김 문제
04장 제어문 이해하기
04-01 if 문
04-02 switch-case 문
04-03 while 문
04-04 for 문
04-05 for each 문
04장 되새김 문제

05장 객체 지향 프로그래밍
05-01 객체 지향 프로그래밍이란?
05-02 클래스
05-03 메서드 더 살펴보기
05-04 값에 의한 호출과 객체에 의한 호출
05-05 상속
05-06 생성자
05-07 인터페이스
05-08 다형성
05-09 추상 클래스
05장 되새김 문제
06장 자바의 입출력
06-01 콘솔 입출력
06-02 파일 입출력
06장 되새김 문제
07장 자바 날개 달기
07-01 패키지
07-02 접근 제어자
07-03 스택틱
07-04 예외 처리
07-05 스레드

07-06 함수형 프로그래밍
07장 되새김 문제
08장 자바 프로그래밍, 어떻게 시작해야 할까?
08-01 내가 프로그램을 만들 수 있을까?
08-02 3과 5의 배수 합하기
08-03 게시판 페이지징하기
08-04 자릿수 구하기
08-05 공백을 제외한 글자 수 세기
09장 자바 코딩 면허 시험 15제
10장 정답 및 풀이
11장 부록
11-01 챗GPT와 함께 자바 공부하기
11-02 public 클래스
12장 마치며



📖 [점프 투 자바 \(/book/31\)](/book/31) / [05장 객체 지향 프로그래밍 \(/218\)](/218) / [05-07 인터페이스 \(/217\)](/217)

🏠 [위키독스 \(/\)](#)

05-07 인터페이스

광고가 출력될 위치입니다.

광고가 출력될 위치입니다.

인터페이스(interface)는 초보 개발자를 괴롭히는 단골손님이다. 인터페이스에 대한 개념 없이 코드로만 이해하려고 하면 곧 미궁에 빠지게 된다. 이렇게 이해하기 힘든 인터페이스는 도대체 왜 필요할까? 새로운 예제를 통해 인터페이스를 차근차근 알아보자.

인터페이스는 왜 필요한가?

인터페이스 작성하기

인터페이스의 메서드

인터페이스 더 파고들기

디폴트 메서드

인터페이스는 왜 필요한가?

다음은 어떤 동물원의 사육사가 하는 일이다.

난 동물원(zoo)의 사육사(zookeeper)이다.
육식동물(predator)이 들어오면 난 먹이를 던져준다(feed).
호랑이(tiger)가 오면 사과(apple)를 던져준다.
사자(lion)가 오면 바나나(banana)를 던져준다.

이와 같은 내용을 코드로 표현해 보자. 먼저 Animal, Tiger, Lion, ZooKeeper 클래스를 작성하자.

Sample.java를 수정해 다음과 같이 코딩해 보자.

```

class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

class Tiger extends Animal {
}

class Lion extends Animal {
}

class ZooKeeper {
    void feed(Tiger tiger) { // 호랑이가 오면 사과를 던져 준다.
        System.out.println("feed apple");
    }

    void feed(Lion lion) { // 사자가 오면 바나나를 던져준다.
        System.out.println("feed banana");
    }
}

public class Sample {
    public static void main(String[] args) {
        ZooKeeper zooKeeper = new ZooKeeper();
        Tiger tiger = new Tiger();
        Lion lion = new Lion();
        zooKeeper.feed(tiger); // feed apple 출력
        zooKeeper.feed(lion); // feed banana 출력
    }
}

```

05-05절에서 보았던 Dog 클래스와 마찬가지로 이번에는 Animal을 상속한 Tiger와 Lion이 등장했다. 그리고 ZooKeeper 클래스를 정의하였다. ZooKeeper 클래스는 tiger가 왔을 때, lion이 왔을 때, 각각 다른 feed 메서드가 호출된다.

ZooKeeper 클래스의 feed 메서드처럼 입력값의 자료형 타입이 다르지만(앞에서는 Tiger, Lion으로 서로 다르다.) 메서드명은 동일하게(여기서는 메서드명이 feed로 동일하다) 사용할 수 있다. 이런 것을 메서드 오버로딩이라고 한다.

프로그램을 실행하면 다음과 같은 결과가 출력된다.

```
feed apple
feed banana
```

만약 Tiger와 Lion뿐이라면 ZooKeeper 클래스는 더 이상 할 일이 없겠지만 Crocodile, Leopard 등이 계속 추가된다면 ZooKeeper는 클래스가 추가될 때마다 매번 다음과 같은 feed 메서드를 추가해야 한다.

다음 코드는 눈으로만 살펴보고 넘어가자.

```
(... 생략 ...)
```

```
class ZooKeeper {
    void feed(Tiger tiger) {
        System.out.println("feed apple");
    }

    void feed(Lion lion) {
        System.out.println("feed banana");
    }

    void feed(Crocodile crocodile) {
        System.out.println("feed strawberry");
    }

    void feed(Leopard leopard) {
        System.out.println("feed orange");
    }
}
```

```
(... 생략 ...)
```

이렇게 동물 클래스가 추가될 때마다 feed 메서드를 추가해야 한다면 ZooKeeper 클래스가 얼마나 복잡해질까? 이런 문제를 해결하기 위해 바로 인터페이스가 필요하다.

인터페이스 작성하기

다음과 같이 코드 상단에 Predator 인터페이스를 추가하자.

```

interface Predator {
}

class Animal {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

(... 생략 ...)

```

이 코드와 같이 인터페이스는 class가 아닌 **interface** 키워드로 작성한다.

인터페이스는 클래스와 마찬가지로 Predator.java와 같은 단독 파일로 저장하는 것이 일반적이나 여기서는 설명의 편의를 위해 Sample.java 파일의 최상단에 작성하였다.

그리고 Tiger, Lion 클래스는 작성한 인터페이스를 구현하도록 다음과 같이 **implements**라는 키워드를 사용해 수정하자.

```

(... 생략 ...)

class Tiger extends Animal implements Predator {
}

class Lion extends Animal implements Predator {
}

(... 생략 ...)

```

이렇게 Tiger, Lion 클래스가 Predator 인터페이스를 구현하게 되면 ZooKeeper 클래스의 feed 메서드를 다음과 같이 변경할 수 있다.

변경전


```
(... 생략 ...)
```

```
class ZooKeeper {
    void feed(Tiger tiger) {
        System.out.println("feed apple");
    }

    void feed(Lion lion) {
        System.out.println("feed banana");
    }
}
```

```
(... 생략 ...)
```

변경후

```
(... 생략 ...)
```

```
class ZooKeeper {
    void feed(Predator predator) {
        System.out.println("feed apple");
    }
}
```

```
(... 생략 ...)
```

feed 메서드의 입력으로 Tiger, Lion을 각각 필요로 했지만 이제 이것을 Predator라는 인터페이스로 대체할 수 있게 되었다. tiger, lion은 각각 Tiger, Lion의 객체이기도 하지만 Predator 인터페이스의 객체이기도 하기 때문에 이와 같이 Predator를 자료형으로 사용할 수 있는 것이다. 05-5절에서 공부했던 IS-A 관계가 인터페이스에도 적용된다. 즉, ‘Tiger is a Predator’, ‘Lion is a Predator’가 성립된다.

- tiger: Tiger 클래스의 객체이자 Predator 인터페이스의 객체

- lion: Lion 클래스의 객체이자 Predator 인터페이스의 객체

다시 말해 Tiger 클래스로 만든 객체는 Tiger 객체이면서 동시에 Predator 객체로도 사용할 수 있다는 의미이다.

이와 같이 객체가 1개 이상의 자료형 타입을 갖게 되는 특성을 다형성(폴리모피즘)이라고 하는데 이는 05-08절에서 자세히 다룬다.

이제 어떤 육식동물 클래스가 추가되더라도 ZooKeeper는 feed 메서드를 추가할 필요가 없다. 다만 육식동물 클래스가 추가될 때마다 다음과 같이 Predator 인터페이스를 구현해야 한다.

여기서 말하는 육식동물 클래스는 Crocodile이나 Leopard와 같이 육식 동물들의 이름을 한 클래스들을 말한다.

```
class Crocodile extends Animal implements Predator {  
}
```

Crocodile 클래스는 실제 코드에 적용하지 말고 눈으로만 살펴보자.

이제 왜 인터페이스가 필요한지 감을 잡았을 것이다. 보통 중요 클래스(ZooKeeper)를 작성하는 시점에서는 클래스(Animal)의 구현체(Tiger, Lion)가 몇 개가 될지 알 수 없으므로 인터페이스(Predator)를 정의하여 인터페이스를 기준으로 메서드(feed)를 만드는 것이 효율적이다.

인터페이스의 메서드

하지만 앞서 살펴본 ZooKeeper 클래스에 한 가지 문제가 있다. ZooKeeper 클래스의 feed 메서드를 보면 tiger가 오든지, lion이 오든지 무조건 "feed apple"이라는 문자열을 출력한다. tiger가 오면 "feed apple"을 출력하는 것이 맞지만 lion이 오면 "feed banana"를 출력해야 한다.

(... 생략 ...)

```
class ZooKeeper {  
    public void feed(Predator predator) {  
        System.out.println("feed apple"); // 항상 feed apple 만을 출력한다.  
    }  
}
```

(... 생략 ...)

```
feed apple  
feed apple
```

이번에도 인터페이스의 마법을 부려 보자. **Predator** 인터페이스에 다음과 같은 **getFood** 메서드를 추가해 보자.

```
interface Predator {  
    String getFood();  
}
```

(... 생략 ...)

여기서 주목할 점이 있다. 메서드에 몸통이 없다는 것이다.

인터페이스의 메서드는 메서드의 이름과 입출력에 대한 정의만 있고 그 내용은 없다. 그 이유는 인터페이스는 ‘규칙’이기 때문이다. 즉, **getFood** 메서드는 인터페이스를 implements한 클래스들이 강제적으로 구현해야 하는 규칙이 된다.

이제 **Predator** 인터페이스에 **getFood** 메서드를 추가하면 **Tiger**, **Lion** 등의 **Predator** 인터페이스를 구현한 클래스에서 컴파일 오류가 발생할 것이다. 오류를 해결하려면 다음처럼 **Tiger**, **Lion** 클래스에 **getFood** 메서드를 구현해야 한다.

(... 생략 ...)

```
class Tiger extends Animal implements Predator {  
    public String getFood() {  
        return "apple";  
    }  
}
```

```
class Lion extends Animal implements Predator {  
    public String getFood() {  
        return "banana";  
    }  
}
```

(... 생략 ...)

Tiger, Lion 클래스의 getFood 메서드는 각각 apple과 banana를 반환하게 했다.

인터페이스의 메서드는 항상 public으로 구현해야 한다.

이어서 ZooKeeper 클래스도 다음과 같이 변경이 가능하다.

(... 생략 ...)

```
class ZooKeeper {  
    void feed(Predator predator) {  
        System.out.println("feed "+predator.getFood());  
    }  
}
```

(... 생략 ...)

feed 메서드가 feed apple 대신 "feed "+predator.getFood() 를 출력하도록 코드를 수정하였다.

predator.getFood() 를 호출하면 Predator 인터페이스를 구현한 구현체(Tiger, Lion)의 getFood() 메서드가 호출된다.

이제 프로그램을 실행해 보자. 원하던 대로 다음과 같은 결과값이 출력되는 것을 확인할 수 있다.

```
feed apple  
feed banana
```

여기서는 왜 인터페이스가 필요한지를 이해하고 넘어가는 것이 가장 중요하다. 동물(Tiger, Lion, Crocodile 등) 클래스의 종류만큼 feed 메서드가 필요했던 ZooKeeper 클래스를 Predator 인터페이스를 이용하여 구현했더니 단 한 개의 feed 메서드로 구현이 가능해졌다. 여기서 중요한 점은 단순히 메서드의 개수가 줄어들었다는 것이 아니다. **ZooKeeper 클래스가 특정 동물 클래스에 의존하던 코드에서 동물 클래스와 무관한 독립적인 코드로 변했다는 것이 핵심이다.** 바로 이 점이 인터페이스의 가장 중요한 장점이다.

이번에는 좀 더 개념적으로 인터페이스를 생각해 보자. 아마도 여러분은 컴퓨터의 USB 포트를 잘 알고 있을 것이다. USB 포트에 연결할 수 있는 기기는 하드디스크, 메모리 스틱, 스마트폰 등 무척 다양하다. 바로 이 USB 포트가 물리적 세계의 인터페이스라고 할 수 있다. USB 포트의 규격만 알면 어떤 기기도 연결할 수 있다. 또, 컴퓨터는 USB 포트만 제공하고 어떤 기기가 연결되는지 신경 쓸 필요가 없다. 바로 이 점이 자바의 인터페이스와 매우 비슷하다. 앞서 만든 ZooKeeper 클래스가 어떤 동물 클래스(Tiger, Lion, ...)이든 상관하지 않고 feed 메서드를 구현한 것처럼 말이다.

물리적 세계	자바 세계
컴퓨터	ZooKeeper
USB 포트	Predator
하드디스크, 메모리 스틱, 스마트폰, ...	Tiger, Lion, Crocodile, ...

점프 투 자바

상속과 인터페이스

Predator 인터페이스 대신 Animal 클래스에 getFood 메서드를 추가하고 Tiger, Lion 등에서 getFood 메서드를 오버라이딩한 후 Zookeeper의 feed 메서드가 Predator 대신 Animal을 입력 자료형으로 사용해도 동일한 효과를 거둘 수 있다. 하지만 상속은 자식 클래스가 부모 클래스의 메서드를

오버라이딩하지 않고 사용할 수 있기 때문에 해당 메서드를 반드시 구현해야 한다는 ‘강제성’을 갖지 못한다. 그래서 상황에 맞게 상속을 사용할 것인지, 인터페이스를 사용해야 할지를 결정해야 한다. 인터페이스는 인터페이스의 메서드를 반드시 구현해야 하는 강제성을 갖는다는 점을 반드시 기억하자.

디폴트 메서드

자바 8 버전 이후부터는 디폴트 메서드(default method)를 사용할 수 있다. 인터페이스의 메서드는 구현체를 가질 수 없지만 디폴트 메서드를 사용하면 실제 구현된 형태의 메서드를 가질 수 있다. 예를 들어 Predator 인터페이스에 다음과 같은 디폴트 메서드를 추가할 수 있다.

```
interface Predator {  
    String getFood();  
  
    default void printFood() {  
        System.out.printf("my food is %s\n", getFood());  
    }  
}
```

디폴트 메서드는 메서드명 가장 앞에 default라고 표기해야 한다. 이렇게 Predator 인터페이스에 printFood 디폴트 메서드를 구현하면 Predator 인터페이스를 구현한 Tiger, Lion 등의 실제 클래스는 printFood 메서드를 구현하지 않아도 사용할 수 있다. 그리고 디폴트 메서드는 오버라이딩이 가능하다. 즉, printFood 메서드를 실제 클래스에서 다르게 구현하여 사용할 수 있다.

디폴트 메서드는 기존 인터페이스에 새로운 기능을 추가해야 할 때 매우 유용하다. 디폴트 메서드를 사용하면 기존에 인터페이스를 구현한 모든 클래스를 수정하지 않고도 새로운 메서드를 추가할 수 있기 때문이다.

마지막 편집일시 : 2025년 8월 20일 5:55 오후

광고가 출력될 위치입니다.

광고가 출력될 위치입니다.

댓글 25 피드백

- 이전글 : 05-06 생성자
- 다음글 : 05-08 다형성

 (/)  